

Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents



01 Numbers



02 Variables or Identifiers



03



04



05

Numbers

- These are numeric values
- Types of numbers in Python:
 - Integers
 - Floating point
 - Complex numbers

Numbers

- Numbers like 5, 5356 are integers
- Numbers like 3.23 and 91.5E-2 are floating point numbers
- And numbers of the form “ $a + bi$ ” are complex numbers

Numbers

- There is no limit to the size of an integer
- Floating point number do have limited precision

Floating Point Number

Floating-point numbers are represented in computer hardware as base 2 (binary) fractions. For example, the decimal fraction

0.125

has value $1/10 + 2/100 + 5/1000$, and in the same way the binary fraction

0.001

has value $0/2 + 0/4 + 1/8$. These two fractions have identical values, the only real difference being that the first is written in base 10 fractional notation, and the second in base 2.

Floating Point Number

Unfortunately, most decimal fractions cannot be represented exactly as binary fractions. A consequence is that, in general, the decimal floating-point numbers you enter are only approximated by the binary floating-point numbers actually stored in the machine.

The problem is easier to understand at first in base 10. Consider the fraction $1/3$. You can approximate that as a base 10 fraction:

0.3

or, better,

0.33

or, better,

0.333

and so on. No matter how many digits you're willing to write down, the result will never be exactly $1/3$, but will be an increasingly better approximation of $1/3$.

Floating Point Number

In the same way, no matter how many base 2 digits you're willing to use, the decimal value 0.1 cannot be represented exactly as a base 2 fraction. In base 2, $1/10$ is the infinitely repeating fraction

```
0.0001100110011001100110011001100110011001100110011...
```

Stop at any finite number of bits, and you get an approximation. On most machines today, floats are approximated using a binary fraction with the numerator using the first 53 bits starting with the most significant bit and with the denominator as a power of two. In the case of $1/10$, the binary fraction is $3602879701896397 / 2^{55}$ which is close to but not exactly equal to the true value of $1/10$.

Floating Point Number

Many users are not aware of the approximation because of the way values are displayed. Python only prints a decimal approximation to the true decimal value of the binary approximation stored by the machine. On most machines, if Python were to print the true decimal value of the binary approximation stored for 0.1, it would have to display

```
>>> 0.1
0.1000000000000000055511151231257827021181583404541015625
```

That is more digits than most people find useful, so Python keeps the number of digits manageable by displaying a rounded value instead

```
>>> 1 / 10
0.1
```

Just remember, even though the printed result looks like the exact value of $1/10$, the actual stored value is the nearest representable binary fraction.

Floating Point Number

Historically, the Python prompt and built-in `repr()` function would choose the one with 17 significant digits, `0.10000000000000001`. Starting with Python 3.1, Python (on most systems) is now able to choose the shortest of these and simply display `0.1`.

Note that this is in the very nature of binary floating-point: this is not a bug in Python, and it is not a bug in your code either. You'll see the same kind of thing in all languages that support your hardware's floating-point arithmetic (although some languages may not *display* the difference by default, or in all output modes).

Floating Point Number

For more pleasant output, you may wish to use string formatting to produce a limited number of significant digits:

```
>>> format(math.pi, '.12g') # give 12 significant digits
'3.14159265359'

>>> format(math.pi, '.2f') # give 2 digits after the point
'3.14'

>>> repr(math.pi)
'3.141592653589793'
```

Literal Constants

- The value of a literal constant can be used directly in programs. For example, 7, 3.9, 'A', and "Hello" are literal constants
- **Numbers** refers to a numeric value. You can use four types of numbers in Python program- integers, long integers, floating point and complex numbers.

Literal Constants

```
>>> 10 + 7  
26
```

```
>>> 50 + 40 - 35  
55
```

```
>>> 12 * 10  
120
```

```
>>> 96 / 12  
8.0
```

```
>>> (-30 * 4) + 500  
380
```

```
>>> 78 // 5  
15
```

```
>>> 78 % 5  
3
```

```
>>> 152.78 // 3.0  
50.0
```

```
>>> 152.78 % 3.0  
2.7800000000000001
```

```
>>> 5**3  
125
```

```
>>> 121**0.5  
11.0
```

Literal Constants

```
>>> 'Hello'  
'Hello'
```

```
>>> "HELLO"  
'HELLO'
```

```
>>> '''HELLO'''  
'HELLO'
```


Arithmetic Overflow

- While multiplying very large floating point numbers you may get an arithmetic overflow
- Try multiplying $2.7e200$ with $4.3e200$
- It shows inf (which means infinity)
- i.e. arithmetic overflow has occurred

Arithmetic Underflow

- While dividing very small floating point number by a very large number you may get an arithmetic underflow
- Try dividing $3.0e-400$ by $5.0e200$
- It shows 0.0
- i.e. arithmetic underflow has occurred

Loss of precision

- When you divide $1/3$ the result is $0.333333...$
- The digit 3 is repeated infinitely
- But we need to take approximate value

Dividing two Integers

- Dividing any two integers produces a floating point number
- Try $4/2$
- It gives 2.0

Multiplying Integer with Float

- You can easily work with integer and floating point number because python automatically converts the integer to a float
- Try `5*3.0`
- It gives 15.0

Quotient & Remainder

- While dividing one number with another if you want to know the quotient and remainder, use floor division (//) and modulo operator (%) respectively
- Try $78//5$, it gives 15
- Try $78\%5$, it gives 3

Exponentiation

- Besides +, -, *, and /
- Python also support ** operator for power of one number to the other
- Try $5^{**}3$, it gives 125

Strings

- String is a group of characters
- Both single quotes ex. `print('Hello')` or double quotes can be used as `print("Hello")`
- Using triple quotes you can write specify multi-line strings

String literal concatenation

- Python concatenates two string literals that are placed side by side
- `print('beautiful weather' '...' 'seems it would rain')`
- It gives: beautiful weather ... seems it would rain

Escape Sequences

- **Some characters (like ", \) cannot be directly included in a string. Such characters must be escaped by placing a backslash before them.**

Escape Sequences

- Try this: `print('What's your name?')`
- It shows syntax error
- You should write: `print('What\'s your name?')`
- Or `print("What's your name?")`
- It gives: `What's your name?`

Escape Sequences

```
>>> print("The boy replies, \"My name is Aaditya.\")  
The boy replies, "My name is Aaditya."
```

Escape Sequence	Purpose	Example	Output
\\	Prints Backslash	print("\\")	\
\'	Prints single-quote	print("\'")	'
\"	Prints double-quote	print("\"")	"
\a	Rings bell	print("\a")	Bell rings
\f	Prints form feed character	print("Hello\fWorld")	Hello World
\n	Prints newline character	print("Hello\nWorld")	Hello World
\t	Prints a tab	print("Hello\tWorld")	Hello World
\o	Prints octal value	print("\o56")	.
\x	Prints hex value	print("\x87")	+

Variables & Identifiers

- Variable means- its value can change
- You can store any piece of information in a variable
- Variables are given appropriate name for easily be identified
- Its value can be used later in the program

Variables & Identifiers

- Variables are examples of Identifiers
- Identifiers are names given to identify something
- Which can be a variable, function, class, module, or other object

Variables & Identifiers

- Rules for naming any Identifier:
 - First character must be a letter (upper or lowercase) or underscore ('_')
 - The rest can be letters, digits, underscore
 - Identifier names are case-sensitive. Ex. myvar and myVar are not the same
 - Character like @, \$, % are not allowed

Variables & Identifiers

- Rules for naming any Identifier:
- Valid: sum, _my_var, num1, r, var_1 etc
- Invalid: 1num, my-var, %check, Basic Sal, H#R&A etc.

Thank You

Dr. Abdul Kayom



Asst. Prof, (SENSE)



kayomabdul@vitap.ac.in



8474860025